Presentation Script: Rotary Double Inverted Pendulum

Introduction

Good morning, everyone. Today, I will be discussing the rotary double inverted pendulum—a complex and fascinating control systems problem that highlights the challenges of stabilizing inherently unstable systems. Our presentation will cover six key areas:

1. The experiment background.
2. System modelling
3. LQR theory
4. Data-driven modeling.
5. The challenges we faced and the solutions we implemented.
6. End results

Let's begin with the experiment background.

Experiment Background

The project that we've been working on is the rotary double inverted pendulum.

It is a benchmark problem in control theory due to its highly nonlinear and unstable nature.

The goal of this experiment is to stabilize the pendulum in its upright position by controlling the rotary arm's motion.

This requires a precise feedback mechanism and careful modelling of the system dynamics.

In this project, we used a combination of linear quadratic regulator (LQR) and data-driven modelling to tackle this tough task.

Data-Driven Modelling

To further enhance the accuracy of the provided model of the rotary double inverted pendulum, we used a data-driven approach. This involved the following steps:

1. **Data Collection and Pre-processing**:
    - With the use of encoders, we've managed to collect angular positions of $\theta, \alpha \; and \; \phi$
    - We then derive velocities and accelerations from these positions by adding integrators in the processing pipeline
    - We also recorded the output voltage of the servo motor and multiplying it with systems electrical and mechanical parameters such as total gear ratio, motor armature resistance, hence getting the torque applied at the base of the rotary arm.

2. **Batch Processing**:
    - Data was divided into uniform batches and shuffled for efficient neural network training.

3. **Neural Network Setup**:

We used a straightforward single-layer perceptron to model and approximate the dynamics of the system.

Before we go into our network, here's a few things worth mentioning.

We were given the initial weight matrix W_0

We also introduced a masking mechanism to ensure that only certain weights are allowed to be updated while improving underperforming connections.

For example, the first column of A will always be 0 as theta has no effect on the behaviours of any other parameters.

We then chose a learning rate of 0.001

Next is the single-layer network

It consists of two parts, one is forward propagation and the other is backward propagation.

Forward propagation is where we pass the inputs x forward and get the dot product of inputs and weight matrix W

As for backward propagation, we compute loss using the mean squared error between predicted Z and true output Y

We then find the gradient of loss which is the derivative of loss with respect to weights.

These gradients guide the direction and magnitude of weight updates.

4. **Training Loop**:

After processing all batches in an epoch, we calculate the average loss across all batches.

This provides a measure of the model's overall performance for that training iteration.

Monitoring epoch loss allows us to track how well the model is learning and identify potential issues, such as overfitting or stagnation.

To improve training stability and ensure convergence, we apply learning rate decay.

Initially, we set a higher learning rate for faster progress, but gradually reduce it by a factor of 0.99 at each epoch.

This helps the model fine-tune its weights in later stages without overshooting the optimal values.

Challenges and Solutions

1. **Data Challenges**:
    - **Unit Discrepancies**: Training data units did not match actual inputs, which required switching to consistent units.
    - **High Background Noise**: Intense noise necessitated the use of low-pass filters.
    - **Sampling Frequency Issues**: A high sampling rate amplified noise, leading us to lower the frequency.
    - **Random Data Set Sizes**: We standardized data collection with fixed runtimes and pre-allocated array lengths.
    - **Inconsistent Filtering**: Sampling and filtering frequencies were aligned to ensure consistent signal processing.
2. **Model Challenges**:
    - **Incorrect Initial Model**: The provided model did not accurately reflect system dynamics. Training revealed diminishing input effects due to errors in the matrix .
    - **Misapplication of Actuator Dynamics**: Errors in actuator dynamics equations were corrected to properly capture the relationship between control inputs and system responses.